总体流程概览

从拿到 HTML 到屏幕显示，大致是这几步（也叫“关键渲染路径”）：

1. **解析构建 DOM 树**
   - 浏览器从网络拿到 HTML 字节流 → 转成字符 → 词法/语法分析 → 构建 **DOM Tree**
   - 遇到 `<script>` 可能会**阻塞解析**，先执行 JS，再继续解析（除非 `defer` / `async`）
2. **解析 CSS，构建 CSSOM 树**
   - 解析所有 CSS（内联、外链、行内样式）→ 构建 **CSSOM（CSS 规则树）**
   - 只有当 **DOM + CSSOM 都准备好** 时，才能进行下一步布局/绘制
3. **生成渲染树（Render Tree）**
   - DOM + CSSOM 合并 → 生成 **Render Tree**
   - 只包含**可见元素**（如 `display: none` 不会进入渲染树）
   - 每个节点都有计算好的最终样式（继承 + 层叠计算后的结果）
4. **布局（Layout / Reflow）**
   - 计算每个渲染树节点的 **几何信息**：位置（x、y）、宽高（width/height）
   - 这是“**重排 / 回流**”：改动会影响周围布局时，就会触发这一步
5. **绘制（Paint / Repaint）**
   - 根据布局结果，把边框、文字、背景、阴影等画成**位图**
   - 这是“**重绘**”：样式改变（颜色、背景等）但不影响布局时，会重新画
6. **分层与合成（Layering & Compositing）**
   - 某些元素会被提升为**独立图层**（如 `position: fixed`、使用 `transform: translateZ(0)`、有 3D 变换、`will-change` 等）
   - 浏览器在 GPU 上对这些图层做**合成（Composite）**：叠加、裁剪、变换
   - 像 `transform`、`opacity` 动画通常只在这一阶段执行，不用重新布局/绘制

与性能相关的几个关键点

- **会触发布局（Reflow）的操作**：改 `width/height/margin/padding/top/left`、插入/删除 DOM、频繁读取布局信息（如 `offsetWidth`）
- **只触发重绘（Repaint）的操作**：改 `color`、`background`、`box-shadow` 等不影响几何的样式
- **只走合成（Composite）的操作**：`transform`、`opacity`（通常是动画里推荐使用的）

简化成一条流水线

可以把上面浓缩成一句话：

HTML/CSS/JS → DOM + CSSOM → Render Tree → Layout（回流） → Paint（重绘） → Composite（合成） → 屏幕